

Split-and-Merge-Based Genetic Algorithm (SM-GA) for LEGO Brick Sculpture Optimization

Publisher: IEEE

[Cite This](#)

PDF

Seung-Mok Lee ; Jae Woo Kim ; Hyun Myung

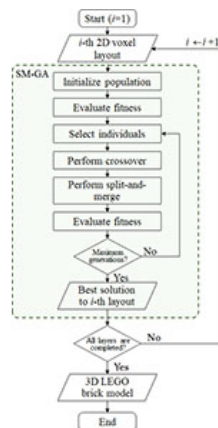
[All Authors](#)11
Cites in
Papers1
Cites in
Patent1285
Full
Text Views

Open Access

Comment(s)

Abstract:

This paper proposes a split-and-merge-based genetic algorithm (SM-GA) for converting a given 3-D voxel model into an LEGO brick sculpture using a minimal number of bricks. The proposed SM-GA is designed to always generate a feasible brick layout in accordance with a given voxel model considering the stability and connectivity between layouts. A novel split-and-merge operator to find the optimal layout is also proposed. To evaluate the effectiveness of the proposed approach, computational and physical experiments are performed. In the computational experiments, the performance of the proposed approach is compared with that of the most recent conventional GA approach. Also, the result of a 3-D physical sculpture made of real LEGO bricks is presented. Compared with the conventional GA-based approach, it is shown that the proposed SM-GA is more effective in finding the near optimal solution to the LEGO brick layout problem.



This figure shows a flowchart describing the overall process including the Split-and-Merge-based Genetic Algorithm (SM-GA) to obtain a 3D LEGO brick sculpture.

Published in: [IEEE Access](#) (Volume: 6)

Page(s): 40429 - 40438

DOI: [10.1109/ACCESS.2018.2859039](#)

Date of Publication: 25 July 2018

Publisher: IEEE

Electronic ISSN: 2169-3536

Funding Agency:

PDF

[Help](#)

SECTION I.

Introduction

LEGO is one of the most popular toys in the world. Various kinds of LEGO bricks can be assembled in many ways to construct such objects as sculptures, buildings, and vehicles. It is, however, very difficult for ordinary users to assemble a desired sculpture without instructions. To create a desired brick sculpture, the stability and connectivity between layers should be considered and the bricks also should be assembled compactly. In this light, there is a demand for a software program that automatically generates an assembly manual to users for a given 3D model [1]– [5].

Generally, the software program performs the processes of approximating a 3D real-world model into a polygonal model and transforming the polygonal model into a 3D voxel model. Conventional approaches divide the 3D voxel model into horizontal 2D layers. The 3D LEGO brick construction problem then simplifies to a series of 2D brick layout problems. The solution to each layer can be stacked together to generate a final 3D LEGO brick sculpture. In this process, the algorithm should provide a feasible solution to each layout. The feasible solution means that there is no conflict between bricks. Also, all filled voxels should be replaced with bricks, and the empty voxel does not have any bricks.

This paper proposes a novel split-and-merge-based genetic algorithm (SM-GA) to create a stable sculpture made of LEGO bricks. To improve the performance of the GA, we propose a new chromosome representation, its initialization method, and a new fitness function considering the connectivity and stability of sculptures. A novel split-and-merge operator is also proposed to effectively reduce the number of bricks and eventually to find a near optimal solution. The proposed SM-GA is designed to always produce feasible solutions in accordance with a given 3D voxel model; i.e., the proposed genetic operators such as crossover and split-and-merge operator always make the solutions feasible during the whole process of the algorithm.

To evaluate the effectiveness of the proposed approach, computational and physical experiments are performed. In the computational experiments, the performance of the proposed SM-GA approach is compared with that of the conventional GA approach proposed in [6]. Also, the results of the physical experiments are presented.

A preliminary version of this paper was presented in [7] with only a 2D simulation result for a single layer. This paper provides more detailed description of the proposed split-and-merge mutation operator with rigorous analysis. This paper also provides a new fitness function with a necessary condition for the weight coefficients to minimize the number of bricks as the number of generation increases.

The rest of this paper is organized as follows. [Section II](#) reviews the previous approaches to 3D LEGO brick construction problem, and [Section III](#) presents the description of the LEGO brick layout problem. [Section IV](#) describes the details of the proposed SM-GA to provide an assembly manual to users, and [Section V](#) presents simulation results and shows actual sculptures from the produced manual. Finally, conclusions are presented in [Section VI](#).

SECTION II. Related Works

To deal with the brick layout problem, greedy algorithm-based approaches were proposed in [8] and [9]. In the proposed approaches, each voxel is filled with a unit brick and then the bricks are merged into larger bricks using a greedy approach. The key drawback of the greedy algorithm-based approaches is that the stability of the whole sculpture is not considered because the fitness is evaluated for each brick individually by considering only its size and connectivity with the bricks directly connected to it.

Some researchers have studied the possibility of applying evolutionary algorithms (EAs) to solve the brick layout problem because they are very effective for combinatorial optimization problems [4], [6], [7], [10]– [13]. The genetic algorithm (GA)-based method proposed by Petrovic [6] is the most recently developed EA to solve the LEGO brick layout problem. The proposed GA is advantageous for enhancing the sculpture stability because each chromosome represents the overall brick placement for each layer. Thus, the fitness is

evaluated for each layer, not for each brick. However, the crossover and mutation operators proposed in the conventional GA [6] are not effective in improving the fitness of the chromosomes because the operators sometimes increase the number of bricks in the layout or produce infeasible brick layouts during the GA's evolutionary process.

An EA combined with a randomized greedy algorithm was also proposed in [4]. The initial brick layout solutions are generated by the randomized greedy algorithm, and the initial solutions are improved by the evolutionary process. During this process, the fitness is evaluated by considering only the number of bricks used in constructing 3D LEGO sculpture and the number of bricks connected with upper and lower layers.

A cellular automata (CA) approach with a cell clustering operation was presented in [3]. The proposed cell clustering operation is to combine small bricks into a larger brick with substantially less memory and execution time.

SECTION III.

Problem Description

The general procedure of automatic brick sculpture generation is as follows: step 1) produce a 3D polygonal model for a given structure; step 2) transform the 3D polygonal model into a voxel model for each layer; step 3) represent the voxel model by a brick sculpture and produce an assembly manual for each layer. The tasks of step 1) and step 2) can be easily done by polygon voxelization and ray-tracing algorithms [1], [8], [14]. However, step 3) is not easy to handle because it is a combinatorial optimization problem in which the stability of the sculpture should be taken into account while using the least number of bricks. This problem is defined as the brick layout problem throughout this paper. Although it appears to be simple, the brick layout problem is, in fact, quite complicated because the solution to the problem should maintain the stability and connectivity using a minimal number of bricks [15].

If the model size is sufficiently large, then a large number of bricks are required, because its interior space should be filled with the bricks. It takes too much time and effort for assembling the model. Thus, it requires a process that deletes voxels in the interior space for saving the bricks. The thickness of the shell should remain properly to keep the stability of the sculpture, considering its size.

In fact, if the interior space of the model is fully occupied by voxels, it is easy to ensure both the connectivity of bricks and the stability of the whole sculpture. Thus, the problem can be solved easily. If the model has a hollow space, however, it is difficult to ensure the stability because the connectivity between layers becomes loose. Therefore, there is a need to find a series of optimal 2D brick layouts, considering the overall stability of the LEGO sculpture and connectivity between layers.

Finally, the brick layout problem can be summarized as follows: find a series of 2D brick layouts, which can guarantee the stability over the 3D sculpture, considering the connectivity between layers that has a hollow space with the least number of bricks. The solution also should be provided within a reasonable time.

SECTION IV.

The Proposed Split-and-Merge-Based Genetic Algorithm (SM-GA)

This section proposes a novel SM-GA to deal with the brick layout problem. The overall process to obtain a stable LEGO brick sculpture can be summarized as follows: divide a given 3D voxel model into multiple layers, which are represented by 2D voxel layouts. After that, perform the proposed SM-GA for each layer. The proposed SM-GA finds an optimized brick layout, considering the connectivity with its lower layer. By repeating this process layer-by-layer, from bottom to top layer, the 3D voxel model can be transformed into a stable sculpture composed of LEGO bricks. Fig. 1 shows a flowchart describing the overall process

including the SM-GA to obtain a 3D LEGO brick sculpture. The proposed SM-GA introduces several new schemes to improve the performance of the conventional GA proposed in [6]. The details are given in the following subsections.

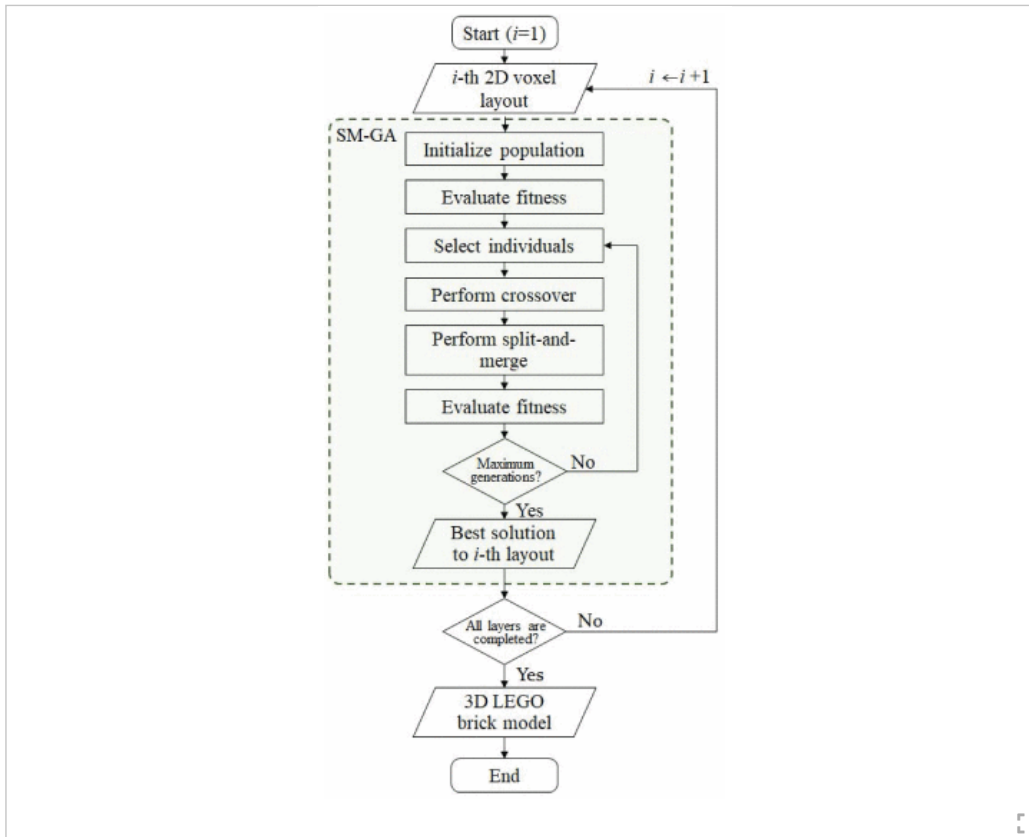


FIGURE 1.

A flowchart of the overall process to obtain a stable LEGO brick sculpture.

A. Chromosome Representation Scheme

We let a chromosome (i.e., candidate solution) contain the brick information for each layer. Each brick layout is encoded by an unordered list of quadruplets (x, y, m, n) , where (x, y) is the position of the upper-left corner of a brick and $(m \times n)$ is the dimension of available brick with m rows and n columns. Fig. 2 shows several brick types used in this paper. These are the standard LEGO brick types, which have the same brick height. Consequently, each layer can be represented by a list of quadruples.

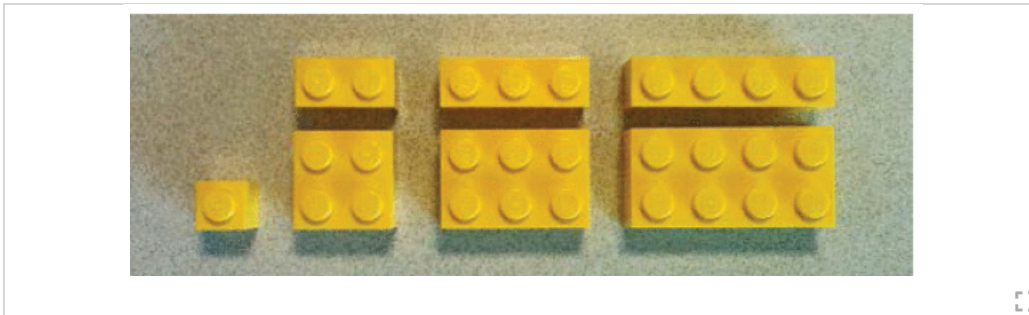


FIGURE 2.

Standard LEGO brick types used in this paper. The bricks have the same height. The bottom right brick has a dimension of $(m \times n) = (2 \times 4)$.

B. Initialization From 2D Voxel Layout

The initial population is randomly generated. Given a $2D$ voxel layout, each chromosome is initialized by the following procedure:

- Step 1: Randomly select a location that is not yet covered by other bricks, and choose a certain direction (i.e., from top to bottom and left to right).
- Step 2: Select one of the bricks that can be inserted at this location, with a larger brick having higher probability to be selected. The selection probability of brick i is defined as

$$\frac{m_i \times n_i}{\sum_{j \in \mathcal{N}} m_j \times n_j}, \quad (1)$$

[View Source](#) 

where m_i and n_i are the numbers of rows and columns of brick i , respectively; and $\mathcal{N}$ is the set of the bricks that can be inserted at the location.

- Step 3: Repeat Step 1 and Step 2 until all $2D$ voxels are covered by bricks.

This initialization method can always provide feasible solutions, which correspond to the given $2D$ voxel layout.

C. Evaluation and Selection

For the connectivity and stability of the $3D$ model, most of the previous works [6], [8], [14], [16] designed the fitness function based on the following factors: (1) the use of the minimum number of bricks; (2) maximizing overlap area of each brick with other bricks in the layers above and below; (3) alternating directionality of the bricks in successive layers.

In this paper, the fitness function is designed to evaluate the fitness of each layer by referring to the three factors as follows:

$$f = c_1 \cdot \frac{1}{n_b} + c_2 \left(1 - \frac{1}{n_u + 1} \right) + c_3 \left(1 - \frac{1}{n_p + 1} \right) \quad (2)$$

[View Source](#) 

where n_b is the number of all bricks in the current layer; n_u is the number of bricks in the lower layer that is connected to the brick; and n_p is the number of bricks that cover the lower layer perpendicularly. The bricks are perpendicular each other if a pair of bricks has opposite directions, i.e., one brick has a vertical direction ($m > n$) and the other brick has a horizontal direction ($m < n$). Only bricks of rectangular types can contribute to the variable n_p . Generally, a smaller value of n_b and larger values of n_u and n_p improve the stability of the LEGO brick sculpture, as reported in [6] and [16]. Thus, the fitness function (2) is designed to minimize n_b and maximize n_u and n_p . c_1 , c_2 , and c_3 are the positive weight coefficients for the three variables, respectively.

Fig. 3 shows an example of how the variables are computed for each layer of the LEGO brick model consisting of 4 bricks. For the lower layer consisting of brick 1 and brick 2, the variables can be computed as $n_b = 2$, $n_u = 0$, and $n_p = 0$. The variables n_u and n_p for the lowest layer are always zero because the lowest layer has no bricks connected to it in the downward direction. For the upper layer consisting of brick 3 and brick 4, the variables can be computed as $n_b = 2$, $n_u = 4$, and $n_p = 4$. Each brick in the upper layer is connected to brick 1 and brick 2, and is also perpendicular to brick 1 and brick 2.

PDF

Help

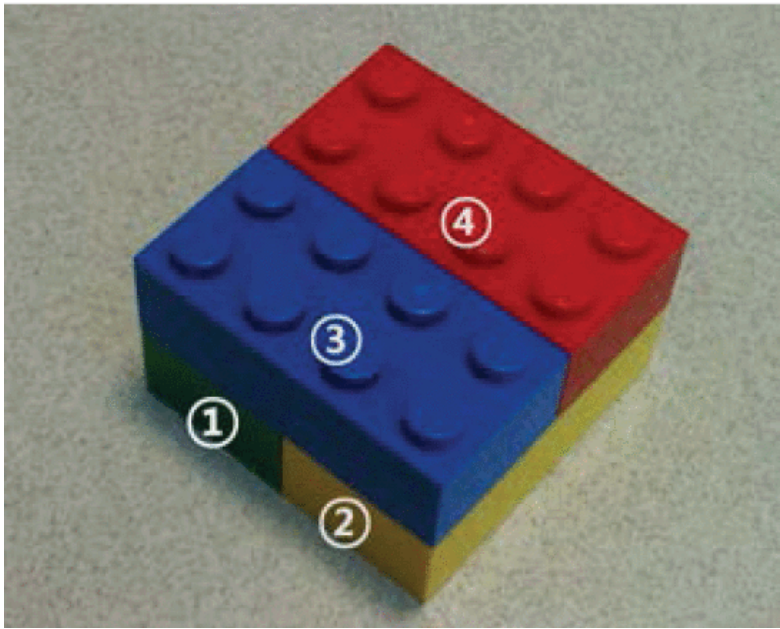


FIGURE 3.

An example of a LEGO brick model consisting of 4 bricks. For the upper layer, each variable in (2) can be computed as $n_b = 2$, $n_p = 4$, and $n_u = 4$.

Minimizing the value of n_b is generally the most effective way to enhance the stability. Thus, a smaller n_b should be prioritized rather than increases in n_u and n_p at each generation. Based on the fitness function (2), this can be represented by the following condition:

$$c_1 \cdot \frac{1}{n_b} + c_2 \left(1 - \frac{1}{n_u + 1}\right) + c_3 \left(1 - \frac{1}{n_p + 1}\right) < c_1 \cdot \frac{1}{n_b - 1}. \quad (3)$$

[View Source](#)

If the coefficients c_1 , c_2 , and c_3 satisfy the condition (3), the fitness value increases regardless of n_u and n_p , even though just one brick is reduced for one generation. Since $1 - \frac{1}{n_u + 1} < 1$ and $1 - \frac{1}{n_p + 1} < 1$, the fitness function (2) satisfies the following inequality:

$$c_1 \cdot \frac{1}{n_b} + c_2 \left(1 - \frac{1}{n_u + 1}\right) + c_3 \left(1 - \frac{1}{n_p + 1}\right) < c_1 \cdot \frac{1}{n_b} + c_2 + c_3. \quad (4)$$

[View Source](#)

Based on (3) and (4), a necessary condition that the decrease in n_b takes priority over the increases in n_u and n_p can be represented as follows:

$$c_1 \cdot \frac{1}{n_b} + c_2 + c_3 < c_1 \cdot \frac{1}{n_b - 1}. \quad (5)$$

[View Source](#)

PDF

Help

The above condition can be represented as follows:

$$c_2 + c_3 < c_1 \left(\frac{1}{n_b - 1} - \frac{1}{n_b} \right). \quad (6)$$

[View Source](#)

If $n_b \geq 2$, then the condition that $0 < \frac{1}{n_b - 1} - \frac{1}{n_b} \leq 0.5$ is always satisfied. Finally, a necessary condition for c_1 , c_2 , and c_3 can be obtained as follows:

$$c_1 > 2 \cdot (c_2 + c_3). \quad (7)$$

[View Source](#)

The weight condition (7) should be satisfied to find the near optimal solution using a minimal number of bricks.

Based on the individual's fitness, a new population is selected to be reproduced in the next generation. This paper uses a rank-based selection method because it was found that this method works better than other methods such as roulette-wheel selection and binary tournament methods for the brick layout problem [6]. In the rank-based selection method, the chromosomes are ordered according to their fitness values. The selection probability of a chromosome is then assigned according to its rank.

D. Crossover

A one-point crossover operator is adopted and modified to fit into the brick layout problem. Referring to Fig. 4, the modified one-point crossover procedure is as follows:

- Step 1: Select two parent chromosomes at random from the population.
- Step 2: Select a crossover direction (horizontal or vertical) and a crossover point.
- Step 3: Divide each parent chromosome into two parts based on the crossover direction and crossover point. The bricks which belong to both parts are included in the upper or left part for convenience. After that, swap the two parts between the parents.
- Step 4: If there are conflicts between bricks, remove the bricks and fill the empty space with random bricks. Similar to the initialization process, a larger brick has a higher probability of being selected.

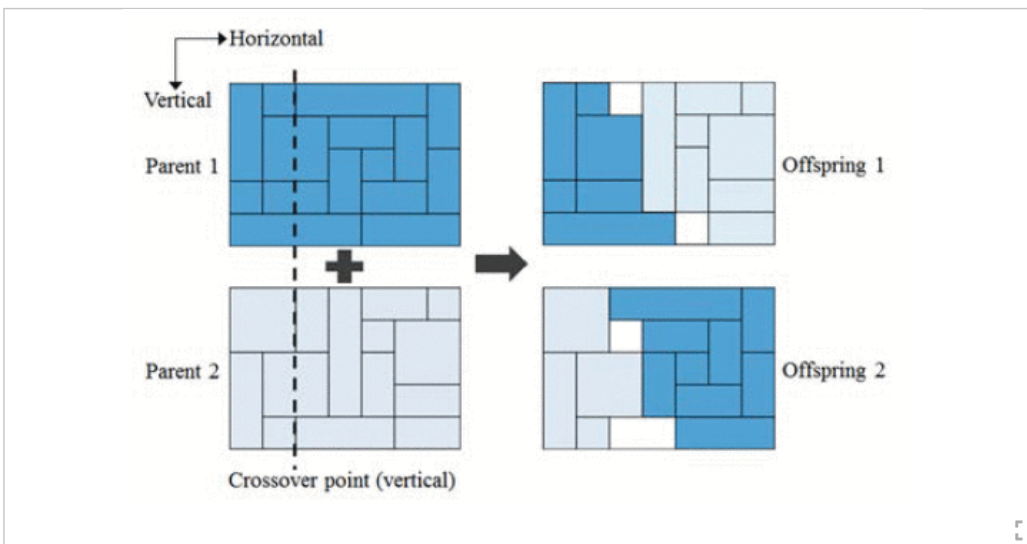


FIGURE 4. The procedure of one-point crossover operator.

Note that the crossover operator may not improve the fitness because the number of bricks remains the same, or increases, during the process of dividing and swapping between two parent chromosomes. The crossover operator, however, is effective to explore other possible candidate solutions.

E. Split-and-Merge Operator

Conventional genetic algorithms generally use a mutation operator to maintain genetic diversity from one generation of a population to the next [17]. Generally, a mutation flips one or more gene values of a chromosome, thereby entirely changing its fitness. In [6], several mutation operators that can be applied to the brick layout problem were proposed, for example, replacing a brick by another random brick, extending a brick by 1 unit in a random direction, shifting a brick by 1 unit in a random direction, eliminating a random brick, merging neighboring bricks into a larger brick, and so on.

However, these conventional mutation operators are not suitable for the chromosome representation introduced in Section IV-A. The mutation operators cannot contribute to increasing the fitness value of the chromosome because they increase the number of bricks in the layer. Fig. 5 shows an example of the conventional shift mutation operator. Applying the shift mutation operator to the layout given in Fig. 5, the conflicts occur between bricks. Thus, all bricks that are located on the way would be eliminated. If we remove some of the bricks to resolve the conflicts, empty space occurs, and the number of bricks increases eventually, in the process of filling in the empty space. As the other conventional mutations such as replacing and extending a brick also produce the conflicts between bricks, the number of bricks increases, in the process of resolving the conflicts. The merge operator does not produce the conflicts between bricks. However, in the case of the brick layout given in Fig. 5(a), any bricks cannot be merged into a larger brick by the conventional merge operator.

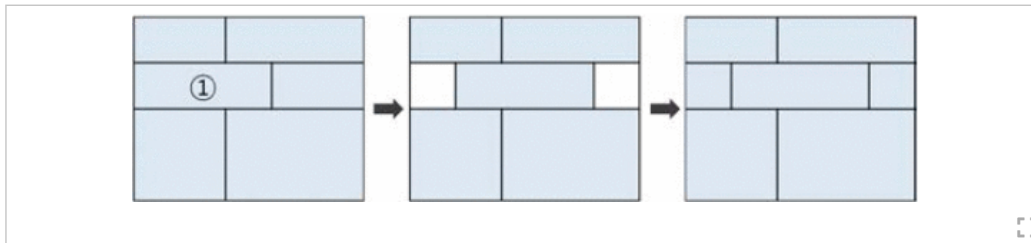
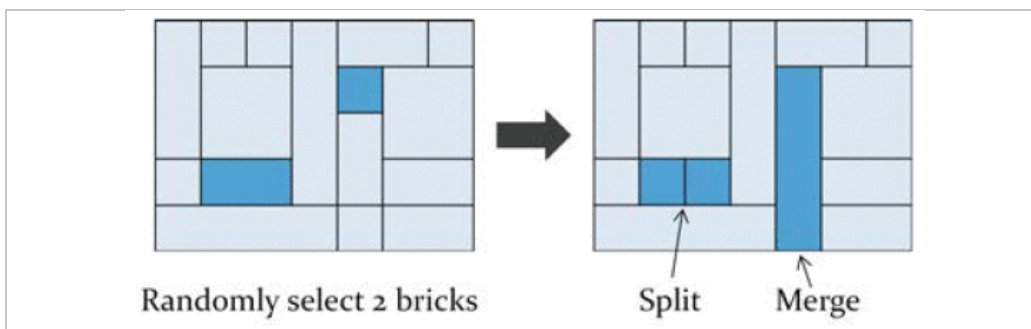


FIGURE 5.

An example of the conventional shift mutation operator. Brick 1 is randomly selected for applying mutation operator randomly (left). Brick 1 moves unit space in right direction, and two empty spaces occur because some bricks in conflict with brick 1 are removed (middle). The empty spaces are filled with random bricks (right).

In order to improve the fitness value of chromosomes, a split-and-merge operator is newly proposed, as described in Fig. 6. The proposed split-and-merge operator is as follows:

- Step 1: Select a brick randomly from a chromosome.
- Step 2: Split the selected brick into 1×1 bricks.
- Step 3: Select another brick randomly.
- Step 4: Merge the selected brick with the largest brick type that can be merged with the neighboring bricks.



**FIGURE 6.**

The procedure of the proposed split-and-merge operator.

At the l -th generation, the probability of the split-and-merge operator, p_l , is determined as follows:

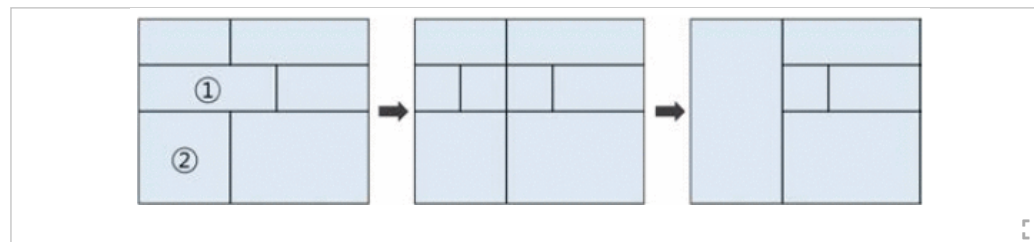
$$p_l = p_{\max} - \frac{p_{\max} - p_{\min}}{L_{\max}} \times l, \quad (8)$$

[View Source](#) 

where $p_{\max}$ and $p_{\min}$ are the upper and lower bounds for p_l ; $L_{\max}$ is the maximum number of generations. The probability of the split-and-merge operator linearly decreases to maintain a balance between the global exploration and the local exploitation as the number of generation increases.

The proposed split-and-merge operator splits one random brick into 1×1 bricks and then merges another random brick with neighboring bricks. The split-and-merge operator can reduce the number of bricks in the brick layout. Also, the bricks can be replaced with other types of brick that can increase n_u and n_p . Thus, the proposed split-and-merge operator effectively improves the fitness of chromosomes.

[Fig. 7](#) shows an example of the proposed split-and-merge operator. The initial layouts shown in [Fig. 7](#) is the same as the initial layout in [Fig. 5](#). Compared with the result of the conventional approaches, the result of [Fig. 7](#) shows that the proposed split-and-merge operator can reduce the number of bricks used in the layout. If the simple merge operation applies to the brick layout, it is highly possible to get stuck in a local minimum, which consists of middle and small size bricks only. However, splitting a brick before merging process can lead to find a near optimal solution by reducing the number of bricks.

**FIGURE 7.**

An example of the proposed split-and-merge mutation operator. Brick 1 and brick 2 are randomly chosen to split and merge, respectively (left). Brick 1 is split into two unit bricks (middle). Brick 2 is merged into 4×2 (right).

In some cases, the proposed split-and-merge operator might be skipped or partially operated. For example, if a unit brick is selected for the split process of Step 1 and Step 2, then the brick is left as it is, because it cannot be split into smaller bricks any more. Similarly, if the largest brick is selected for the merge process of Step 3 and Step 4, the brick remains unchanged.

SECTION V. Results

A. Experiments for 2D Layouts

To validate the effectiveness of the proposed SM-GA approach, the performance of the proposed approach is compared with that of the conventional GA-based approach proposed in [6] through computational experiments. For the computational experiments, the population size is set to 50, and the maximum number of generations $L_{\max}$ is limited to 1,000. The upper and lower bounds for the probability of the split-and-

PDF

Help

merge operator in (8) are set to $p_{\max} = 0.7$ and $p_{\min} = 0.1$, respectively. The weight coefficients in (2) are set to $c_1 = 5$, $c_2 = 1$, and $c_3 = 1$, which satisfy the condition (7).

Fig. 8 shows a 3D pyramid type polygonal model and its voxel model. The polygonal model is transformed into a discrete set of voxels using the vtkImplicitModeller class included in the visualization toolkit (VTK) [18]. The voxel model consists of 13 layers with a voxel size of 8×8 . Fig. 9 shows the layers from the 5th to the 8th of the voxel model shown in Fig. 8(b). The cell denoted by '0' means that the voxel is empty, and the gray block denoted by '1' means that the voxel is filled. Because the inner voxels are removed to reduce the number of bricks, the inner voxel of each layer is represented with empty cells, as shown in Fig. 9. From the bottom layer to the top layer, each layer is processed by the proposed SM-GA considering the connectivity with its lower layer.

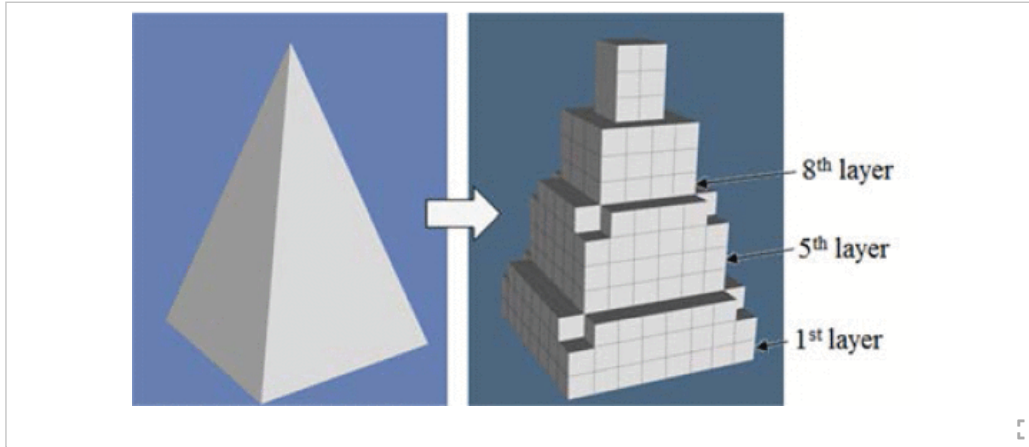


FIGURE 8.

A pyramid model used in experiments. 3D input polygonal model (left) and its voxel model (right). The voxel model has 13 layers and each layer has 8×8 voxel size.

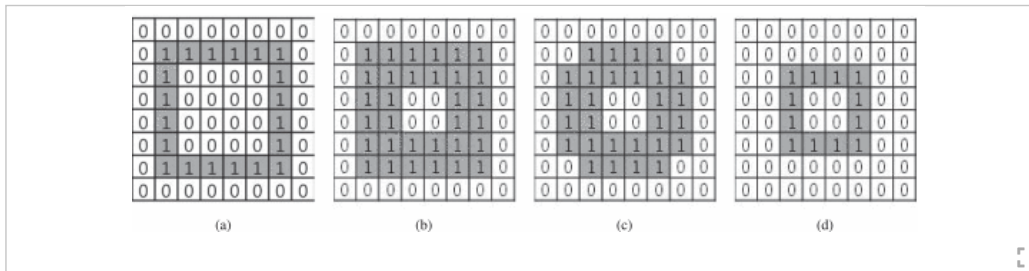
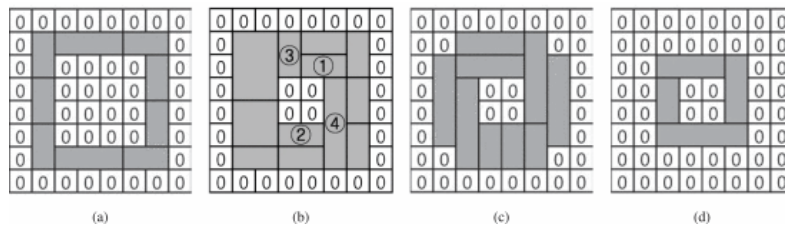


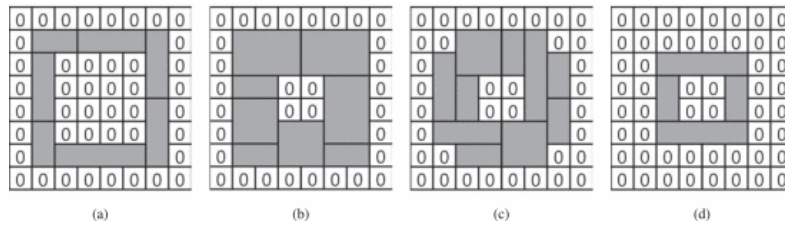
FIGURE 9.

Input voxel layouts. The white cell denoted by '0' means that the voxel is empty, and the gray cell denoted by '1' means that the voxel is filled. (a) 5th-layer. (b) 6th-layer. (c) 7th-layer. (d) 8th-layer.

Figs. 10 and 11 show the results from the conventional GA and the proposed SM-GA. From these results, it is found that the layouts produced by the proposed SM-GA are more stable than the results produced by the conventional GA. For example, in Fig. 10(b), brick 1 and brick 2 on the 6th layer are not connected to any other bricks on the 5th layer. Also, brick 3 on the 6th layer is connected with a unit cell of the 5th layer. Although these bricks can be assembled with the upper layer, it will make the overall brick model less stable. However, all bricks on the layouts produced by the proposed SM-GA are directly connected to the lower layers.

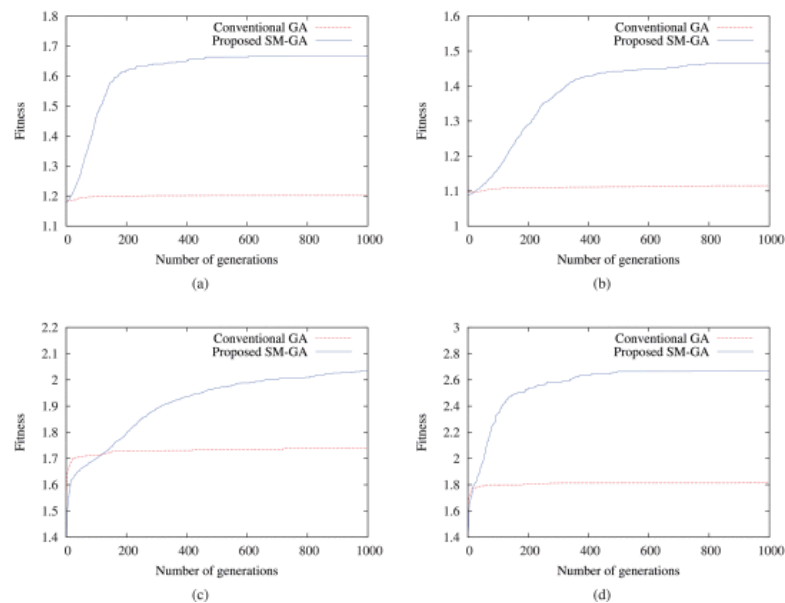
**FIGURE 10.**

Results of the conventional GA-based approach. Brick 1 and brick 2 on the 6th layer are not connected to any other bricks on the 5th layer. Also, brick 3 on the 6th layer is connected with a unit cell of the 5th layer. These make the overall brick model less stable. (a) 5th-layer. (b) 6th-layer. (c) 7th-layer. (d) 8th-layer.

**FIGURE 11.**

Results of the proposed SM-GA approach. All bricks on the layouts are directly connected to the lower layers. (a) 5th-layer. (b) 6th-layer. (c) 7th-layer. (d) 8th-layer.

Fig. 12 shows the average fitness defined in (2) for 30 runs. The fitness of the conventional GA increases slightly in the beginning, but it converges before 100 generations. However, the fitness of the proposed SM-GA converges to a much higher value as the number of generations increases, compared with the conventional GA approach.

**FIGURE 12.**

Mean values of the fitness defined in (2) for 30 runs. (a) 5th-layer. (b) 6th-layer. (c) 7th-layer. (d) 8th-layer.

B. Experiments for 3D Model

A physical experiment is also performed with the 3D pyramid model. Fig.13 shows the process of building an actual sculpture using real LEGO bricks. The sculpture is constructed based on the layouts generated by the proposed SM-GA approach. From the physical experiments, it is found that the bricks are well connected with each other and the sculpture is well stabilized.

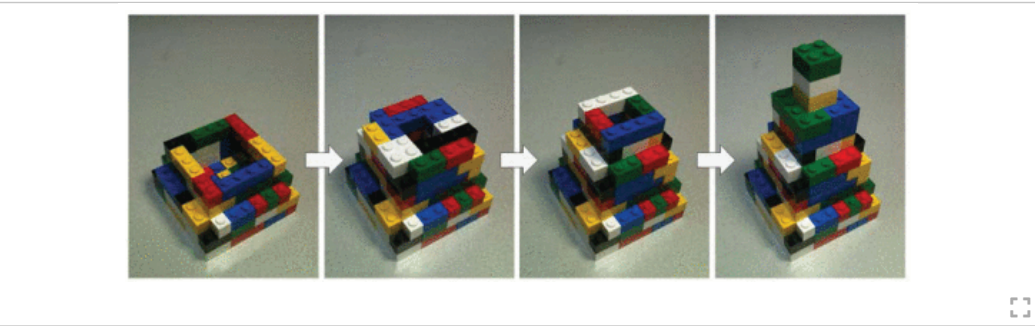


FIGURE 13. Actual sculpture model of real LEGO bricks. The bricks of the sculpture are well connected with each other, and the sculpture is well stabilized.

In addition, computational experiments are performed with bigger and various 3D models. Table 1 shows the dataset used in the experiments, indicating the model name, dimension, and the total number of filled voxels denoted by N_v . Figs.14– 17 show the LEGO brick sculpture models generated by the proposed SM-GA. Through the 3D computational experiments, we have found that the SM-GA generates feasible solutions that have no conflicts between bricks.

TABLE 1 The Size of Each 3D Model Used in the Experiments

Name	Dimension (width × height × layers)	Number of voxels (N_v)
Pyramid	$8 \times 8 \times 13$	324
Gourd	$14 \times 31 \times 19$	723
Icosahedron	$17 \times 17 \times 17$	968
Male	$23 \times 58 \times 54$	5,106
Space shuttle	$146 \times 55 \times 93$	13,222

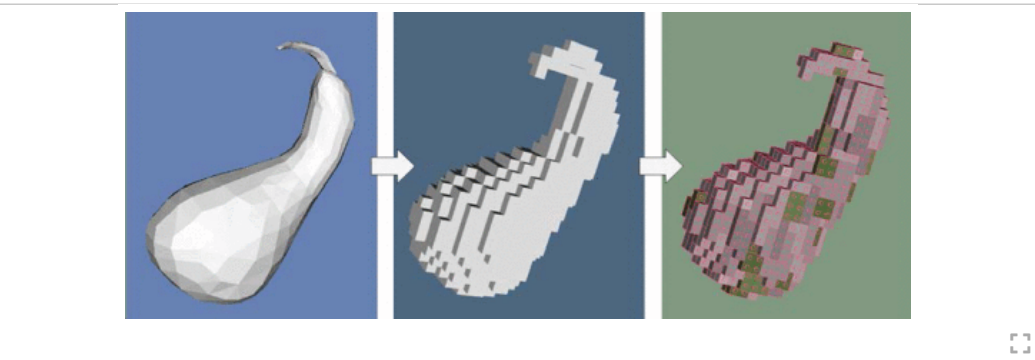


FIGURE 14. Computational results of a gourd model. A 3D input polygonal model (left), its voxel model (middle), and a LEGO brick model (right).



FIGURE 15. Computational results of a icosahedron model. A 3*D* input polygonal model (left), its voxel model (middle), and a LEGO brick model (right).

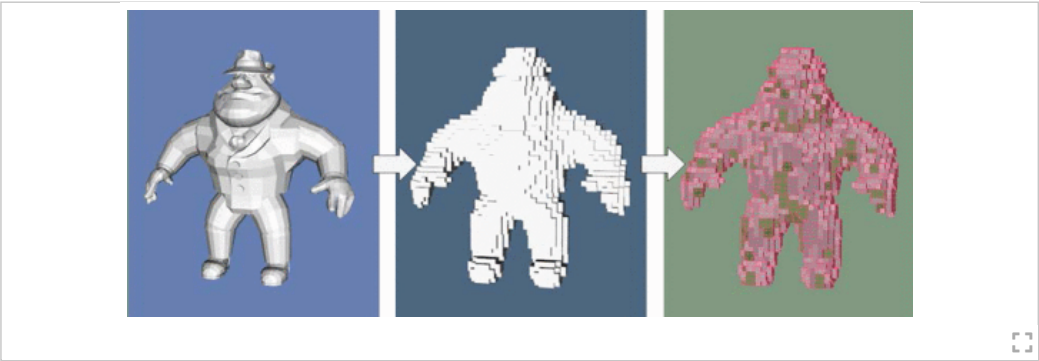


FIGURE 16. Computational results of a male model. A 3*D* input polygonal model (left), its voxel model (middle), and a LEGO brick model (right).



FIGURE 17. Computational results of a space shuttle model. A 3*D* input polygonal model (left), its voxel model (middle), and a LEGO brick model (right).

Table 2 shows the experimental results including the total number of used bricks, the voxel-to-brick ratio, and the improvement percentage for each 3*D* model. The number of used bricks and the value of voxel-to-brick ratio are the mean values for 30 runs. The voxel-to-brick ratio, which is the number of voxels divided by the number of bricks, shows how effective the output layouts are in reducing the number of bricks. The improvement percentage is defined as $\frac{N_c - N_p}{N_c}$, where N_c and N_p are the number of bricks generated by the conventional and proposed approaches, respectively. The higher the voxel-to-brick ratio or the improvement percentage, the better the performance.

TABLE 2 Comparison Results Between the Conventional GA and the Proposed SM-GA. The Number of Used Bricks and the Value of Voxel-to-Brick Ratio are the Mean Values for 30 Runs

--

PDF

Help

3D Model	Conventional GA		Proposed SM-GA		Improvement percentage (%) $\left(\frac{N_c - N_p}{N_c} \times 100\right)$
	Number of bricks (N_c)	Voxel-to-brick ratio $\left(\frac{N_v}{N_c}\right)$	Number of bricks (N_p)	Voxel-to-brick ratio $\left(\frac{N_v}{N_p}\right)$	
Pyramid	107.2	3.022	101.0	3.208	5.784
Gourd	267.5	2.703	253.0	2.858	5.421
Icosahedron	357.8	2.705	340.5	2.843	4.835
Male	1,784.8	2.861	1,741.0	2.933	2.454
Space shuttle	4,078.7	3.241	4,027.3	3.283	1.260

The results show that the proposed split-and-merge mutation operator uses a small number of bricks to create the LEGO brick sculpture for all given models. The voxel-to-brick ratio value depends on the complexity of the layout. Therefore, the space shuttle model shows the highest voxel-to-brick ratio value because of its relatively low complexity, even though the space shuttle 3D model has the largest number of voxels among the given five models. This implies that SM-GA can perform well regardless of the size of 3D model.

However, the improvement percentage decreases as the voxel number of the 3D model increases. This is because the brick size that can be merged at once by the proposed mutation operator is limited to the bricks shown in Fig. 2. The effect of the proposed mutation operator can be large for a small 3D model, but the effect is relatively small for a large 3D model.

SECTION VI.

Conclusion

This paper proposed a novel SM-GA to create a stable sculptures made of LEGO bricks. The purpose of the algorithm is to automatically generate an assembly manual so that the users can easily assemble desired sculptures. The performance of the proposed SM-GA was verified through computational and physical experiments. The main contributions of this paper can be summarized as follows: First, a novel split-and-merge operator was proposed to reduce the number of bricks and to find a near optimal solution. In particular, it was shown that the proposed operator is more effective in reducing the number of bricks compared with conventional mutation operators through various computational experiments; second, a new fitness function was proposed with a necessary condition for the weight coefficients to minimize the number of bricks as the number of generation increases. The necessary condition was mathematically proven to be effective in reducing the number of bricks, and this is a new result not found in other EAs; third, the proposed SM-GA is designed to always generate a feasible brick layout in accordance with a given 3D voxel model considering the stability whereas the conventional GA-based approaches produce infeasible brick layouts in most cases. Compared with the most recent conventional GA-based approach, it was shown that the proposed SM-GA is more effective in reducing the number of bricks and finding near optimal solutions to the brick layout problem.

Authors



Figures



References



Citations



Keywords



Metrics



PDF

Help

ALSO ON IEEE XPLORE

UETT4K Anti-UAV: A Large Scale 4K ...

3 months ago · 1 comment

In recent years, incidents involving unmanned aerial vehicles (UAVs) have ...

A Computational Intelligence ...

5 months ago · 1 comment

This research presents a robust and comprehensive framework for predicting ...

X-SPIDE: An eXplainable Machine Learning ...

2 months ago · 1 comment

Blockchain technology is revolutionizing digital asset exchange by eliminating ...

Answer Distillation Network With ...

6 months ago · 1 comment

Medical Visual Question Answering (Med-VQA) is a multimodal task that aims ...

Research on SNN Learning Algorithms

3 months ago · 1 comment

Spiking Neural Networks inspired by the brain's neuronal information ...

0 Comments

Login ▾

G

Start the discussion...

LOG IN WITH



OR SIGN UP WITH DISQUS ?

Name

Email

Password

This comment platform is hosted by Disqus, Inc. I authorize Disqus and its affiliates to:

- Use, sell, and share my information to enable me to use its comment services and for marketing purposes, including cross-context behavioral advertising, as described in our [Terms of Service](#) and [Privacy Policy](#), including supplementing that information with other data about me, such as my browsing and location data.
- Contact me or enable others to contact me by email with offers for goods or services
- Process any sensitive personal information that I submit in a comment. See our [Privacy Policy](#) for more information

→

♡

Share

Best Newest Oldest

Be the first to comment.

Subscribe

Privacy

Do Not Sell My Data

IEEE Personal Account

CHANGE USERNAME/PASSWORD

Purchase Details

PAYMENT OPTIONS

VIEW PURCHASED DOCUMENTS

Profile Information

COMMUNICATIONS PREFERENCES

PROFESSION AND EDUCATION

TECHNICAL INTERESTS

Need Help?

US & CANADA: +1 800 678 4333

WORLDWIDE: +1 732 981 0060

CONTACT & SUPPORT

Follow

PDF

Help

